

Android System Integrity: Comparing Key Attestation and the Play Integrity API

Bernhard Schertler
bernhard.schertler@pm.me
Johannes Kepler University
Linz, Austria

Abstract

This paper presents an overview of Android’s key attestation and Play Integrity API systems, examining their roles in device and application security. We evaluate both approaches, with additional focus on their impact on third-party Android distributions. Our analysis reveals that while the Play Integrity API offers robust security features and is very simple to implement, its proprietary nature, reliance on Google Play Services and not certifying 3rd party operating systems raise anti-competitive concerns. The hardware attestation API on which the Play Integrity API is built upon provides a more flexible alternative, enabling support for third-party operating systems while maintaining strong security guarantees through hardware-backed key storage. We also examine different solutions like GrapheneOS’s Auditor application, demonstrating an alternative approach to platform attestation, as well as serving as a sample of using the hardware attestation API. Our findings suggest that while the Play Integrity API is a solid security tool, its reliance on proprietary Google systems may hinder competition and user choice in the Android ecosystem, especially since it is mandatory for the use of certain Apps.

CCS Concepts

• **Security and privacy** → **Software and application security**;
Mobile platform security; *Trusted computing*.

Keywords

android, play integrity API, key attestation, platform attestation, system integrity

1 Introduction

As modern smartphones have evolved to become integral parts of everyday life, including sensitive information and financial data, maintaining a secure system is highly important. To prevent modified systems from going undetected, integrity systems have been developed to ensure a trusted system state. However, these efforts have been mostly focused on proprietary Google services, starting with SafetyNet and not Play Integrity API. This leads to issues for 3rd party operating systems, especially with hardware attestation becoming required by more and more Apps.

2 Play Integrity API

The Play Integrity API is the current implementation by Google of an attestation system to verify a secure system state. It supersedes the older SafetyNet Attestation API, expanding and improving the integrity checks to optionally make use of modern hardware based attestation. [12]

2.1 SafetyNet

The SafetyNet API was Google’s initial solution for Android device integrity verification, providing developers with a mechanism to assess the security state of devices running their applications and prevent Apps from running on modified systems. The API has since started being phased out since 2022 in favor of the Play Integrity API, with a total shutdown in January 2025 [1].

2.2 Core Functionality and Architecture

The Integrity API performs three critical verification checks in parallel:

- (1) Device integrity verification to detect signs of system compromise
- (2) App integrity validation to ensure the application hasn’t been tampered with
- (3) Play Store delivery verification to confirm legitimate app distribution

The API implements a challenge-response protocol where the client device requests an integrity verdict from Google’s servers. This verdict contains cryptographically signed attestations about the device’s state, the application’s authenticity, and its distribution channel [12].

2.3 Device Integrity Verification

The device integrity component examines multiple system parameters to detect potential security compromises. It verifies:

- Boot state integrity through bootloader checks
- System image authenticity
- Runtime environment security
- Presence of known security threats

Unlike its predecessor SafetyNet, the Integrity API employs harder to bypass detection mechanisms, relying on hardware support to prevent any software bypasses. However, enforcing this stricter attestation is optional and up to developers. Currently, enforcement of this is still rare due to legacy devices but is likely to become more widespread as time goes on. [12]

2.4 Application Integrity Validation

The application integrity verification ensures that the executing application code matches the version published on the Play Store. This involves:

- Verification of app signing certificates
- Detection of code modifications
- Validation of app packaging
- Resource integrity checking

The API provides granular feedback about specific integrity violations, enabling developers to implement appropriate security responses, although the actual implementation of these measures is often severely lacking, often allowing for bypasses [9].

2.5 Integrity verdicts

Upon completing the Integrity checks, the *Device Integrity* field returns information about the level of integrity that has been met. As stated by the Play Integrity Documentation [13]:

MEETS_DEVICE_INTEGRITY:. Running on Android with Google Play Services. Passes system integrity checks and meets Android comparability requirements.

MEETS_VIRTUAL_INTEGRITY:. Android emulator with Play Services meets integrity checks.

MEETS_BASIC_INTEGRITY:. Passes basic system integrity checks and if running Android 13 or later required Android Platform Key Attestation. Does not require Android compatibility requirements to be met or Google Play Services to be approved. Will pass on devices with unlocked bootloader, unverified boot or devices not certified by the manufacturer.

MEETS_STRONG_INTEGRITY:. Running on Android with Google Play Services. Passes system integrity checks, meets Android comparability requirements and has hardware backed proof of boot integrity. For Android 13 and later also requires a security update within the last year.

2.5.1 Issues with requiring Strong Integrity. While there are barely any Apps requiring Strong Integrity as of now, certain Apps will inevitably opt for requiring it in the near future, once the vast majority of phones in use support it. While this does ensure maximum security, this also comes with several disadvantages locking users out of the Application.

- Preventing users of 3rd party operating systems
- Preventing users of rooted devices
- Preventing sideloading using .apk files

2.6 App Integrity

Similar to Device Integrity, there is also an option to check for App integrity. This checks if the App certificate matches the Version in the Google Play Store, allowing to block potentially malicious modifications by 3rd parties, while still allowing the App to be distributed in other App-Stores and as .apk-Packages.

- *PLAY_RECOGNIZED*: The app and certificate match the versions distributed by Google Play.
- *UNRECOGNIZED_VERSION*: The certificate or package name does not match Google Play records.
- *UNEVALUATED*: Application integrity was not evaluated. A necessary requirement was missed, such as the device not being trustworthy enough.

2.7 License verdicts

Since 2024, the Play Integrity API also supports license verdicts.

- *LICENSED*: The user has an app entitlement, meaning the App was installed directly from the Google Play Store.

- The user doesn't have an app entitlement, meaning that the App was not installed from the Google Play Store.
- *UNEVALUATED*: Licensing details were not evaluated because a necessary requirement was missed. Could be because of missing device integrity, no Play Store being present on the device,...

Developers can prevent the App from running if it is not *LICENSED*, or block certain features.

A license verdict of *LICENSED* ensures that the App was downloaded from the Google Play Store, completely blocking alternative App Stores and manual installations from receiving this verification. This has already seen use in a number of commercial Apps. While this does serve as an easy to implement and reliable form of DRM, as with any form of DRM it also inhibits the freedom of the user. It is also a further competitive disadvantage to alternative App stores due to Google's already quasi-monopoly on App distribution on Android.

2.8 Anti-competitive Aspects

The Play Integrity API's proprietary nature has raised anti-competitive concerns, particularly from GrapheneOS developers. While Google justifies API restrictions through Compatibility Test Suite (CTS) and Compatibility Definition Document (CDD) compliance, there is an indication of inconsistent enforcement favoring Google's business partners [5] [8]. GrapheneOS has documented cases where certified Android devices fail important CTS tests, yet retain their certification and API access, while alternative distributions meeting much higher security standards are excluded. This selective enforcement has led to regulatory concerns and legal threats from GrapheneOS developers, who argue the API serves more as a market control mechanism than a security measure, instead encouraging the use of the hardware attestation API, upon which core parts of Play Integrity is built upon [6] [8].

2.9 Limitations and Considerations

While the Integrity API provides very easy to implement and robust security verification, it has some limitations:

- Only official Android builds are supported, excluding 3rd party operating systems.
- It needs Play Services to work
- Network connectivity is required for verification
- Prevents Apps using it from running on secure, but unofficial builds (GrapheneOS, LineageOS,...)

3 Attestation

Attestation is done remotely using the API's attest function[9]. Precise details about the attestation process is unknown, as this functionality is proprietary and closed source. However, through reverse engineering efforts, the main aspects have been understood. Before verification, the API downloads the binary blobs from the Google servers. No part of the attestation binary is stored within an APK, the verification relies solely on the freshly downloaded binaries. This allows for keeping the attestation process up to date without relying on system or play services updates, which can often be delayed. Thus, ensuring that a vulnerability found is able

to be quickly patched for every device, independent of system or application updates [10].

The attestation process makes use of multiple layers of verification and security checks. When an application initiates an attestation request, the API performs a series of integrity checks on both the device and the requesting application. These checks include:

- Runtime environment verification to detect potential virtualization or emulation
- System integrity validation including bootloader state and system partition modifications
- Application package verification to ensure app integrity
- Hardware-backed key attestation if available on the device

3.1 Software based attestation

Until Android 8, software based attestation has been the default method for key attestation. Since Android 8, every Android device has to offer hardware-based attestation in order to be a certified Android device [2]. Although this does provide basic protection against attacks, the software-based method means that it can be defeated on rooted devices (Magisk Denylist [11], Safetynet-Fix[14],...). While this can be behavior desired by the user in some cases and enables running certain Apps on systems with custom ROMs or when rooted that otherwise block them, it also poses a security risk, as malicious actors could also potentially bypass the attestation and run sensitive Apps (Banking, national ID, ...) on compromised devices.

Software based attestation is still used in the Play Integrity API and is sufficient for *MEETS_BASIC_INTEGRITY* system integrity [13].

3.2 Hardware backed attestation

As all software based solutions are possible to bypass with root level access, the need for hardware based attestation arises. In recent years, this has been implemented into Android, with support being mandatory for new devices shipping with Android 8 or later [3]. This attestation relies on keys stored in the isolated safe environment, thus providing no way for software to interfere. Passing hardware attestation grants *MEETS_STRONG_INTEGRITY* if the device has also received an update within the last 12 months.

As of now, no critical vulnerabilities are known. However, as it is still not widely supported, there does exist the possibility of blocking hardware based attestation and forcing a fall back to software based attestation which can then be bypassed. This bypass will remain possible, as long as Apps do not mandate hardware attestation.

4 Android Hardware Attestation API

Androids Hardware Attestation API is an API to ensure that keys used in Applications are stored in the devices hardware-backed key-store. This ensures that the keys cannot be manipulated by software, even if malicious actors gain root privileges.

This serves as the backbone for the strong integrity of the Play Integrity API, but can also be used on it's own to verify system integrity. It serves as an alternative way to ensure the integrity of the system and the application to the proprietary Play Integrity API [3]. Although this does require some additional effort, like

managing valid keys and does not verify any other things about the system, this does come at the advantage of increased flexibility and not relying on the monopoly of Google.

To verify system integrity, app developers can verify the boot keys against known public keys of the operating system. This includes both official vendor images, as well as 3rd party ones like GrapheneOS. Public keys of images are made available by the operating system developers.

Using this API, developers can whitelist trusted third-party operating systems, allowing users running other systems like GrapheneOS to use their application while still ensuring system integrity. This API has gained particular prominence through its recommendation by security-focused Android distributions like GrapheneOS, offering a more flexible and open approach compared to alternatives such as Google's Play Integrity API.

4.1 Usage

Check for hardware key attestation support: Hardware attestation support is given if the attestation certificate chain has a Google root-signed certificate and that the `attestationSecurityLevel` is set to either `TrustedEnvironment` or `StrongBox`. Additionally, developers must verify all signatures in the certificate chain are valid and check that no keys in the chain have been revoked by consulting the certificate revocation status list. The attestation should only be trusted if the root key matches Google's attestation root key and all these conditions are met [3].

Verifying boot state. To check that the system image is uncompromised, a check on `androidboot.verifiedbootstate` can be used. If the state is *Green/Verified* an official image is run, if it's *Yellow/SelfSigned* there is a 3rd party image being run. Applications can check against known public keys published by OS developers to verify that it is indeed a trusted image [4]. *Red/Unlocked* represents an unlocked bootloader, for which no guarantees can be made, as anyone with physical access can modify the image.

Verification of the certificate chain depends on the root certificate, provided by the device manufacturer or OS vendor (Google, Samsung, Xiaomi, LineageOS, GrapheneOS).

4.2 Certificate revoking

In order to act upon private key leaks, making anyone able to sign any code with the root certificate, the system needs a way of revoking certificates. For this purpose, Google maintains the Certificate Revocation Status List (CRL), storing a list of all revoked certificate keys reported so far [3]. In case of data breaches leaking root certificate keys, they are added as soon as possible to this list in order to revoke them.

4.3 Comparison with Play Integrity API

While Google's Play Integrity API incorporates the hardware attestation API for its strong integrity level checks, it implements a more restrictive approach. Key differences include:

- **Operating System Support:** The hardware attestation API enables developers to support alternative operating systems, whereas Play Integrity API typically requires official Android builds.

- **Dependencies:** Hardware attestation operates independently of Google Play Services, making it more suitable for devices without Google services.
- **Flexibility:** Applications using hardware attestation directly can implement custom verification logic for OS patch levels and other security properties.

5 GrapheneOS Auditor: App based Attestation System

A different approach is taken by GrapheneOS's auditor application. It directly informs the user about device integrity, instead of relying on applications to check the system. Auditor is also based on the hardware attestation API and serves as a good example of how to implement attestation on Android without relying on Play Services. The only part interacting with Google servers is periodically fetching their key revocation list [7] [6].

5.1 Attestation

This tool relies instead of the applications, on the device user and serves as a tool for the user to ensure integrity of their own device. Attestation can be done both remotely using a server (with the option to perform periodic checks) or a secondary Android device. In both cases, the verification process functions in the same way.

The system implements a Trust On First Use (TOFU) model through a pairing mechanism between two devices:

- An Auditor device performing the verification
- An Auditee device undergoing the attestation process

To implement TOFU, Auditor uses an initial verification process chained to a known root certificate.

The attestation process generates signed results containing system properties such as boot keys, system state, security level, patch level, certificates and other system information.

5.1.1 Local Verification. Device verification can be performed locally using a secondary Android device running in Auditor mode. This approach provides immediate verification of device integrity without requiring any internet connectivity, however this does rely on the secondary device being in a trustworthy state and is rather impractical for daily use.

5.1.2 Remote Attestation. Remote verification is facilitated through a server run by the Auditor team (<https://attestation.app/>), which offers scheduled verifications and email alerts in case of issues. Attestation results can also be downloaded [7].

5.2 Strongbox Integration

For devices running Android 9 or later, Auditor can leverage Strong-Box Keymaster implementation, providing enhanced security through:

- Dedicated Hardware Security Module storage for attestation keys
- Reduced attack surface compared to traditional Trusted Execution Environment storage
- Isolation from the main processor's security domain

6 Conclusion

The Play Integrity API and Hardware Attestation API each offer unique benefits and challenges. The Play Integrity API provides strong security and is easy to use, but it also relies heavily on proprietary Google systems, limiting options for users and developers using alternative operating systems like GrapheneOS.

In contrast, using the Hardware Attestation API directly offers more flexibility and supports third-party systems while maintaining strong security. Tools like GrapheneOS's Auditor show that open attestation solutions are possible without depending on Google's services.

Ultimately, the decision between these systems depends on whether developers prioritize ease of integration or flexibility and wider platform support.

References

- [1] About the SafetyNet Attestation API deprecation 2024. <https://developer.android.com/privacy-and-security/safetynet/deprecation-timeline> Retrieved 04.01.2025.
- [2] Android 8.0 Compatibility Definition 2024. <https://source.android.com/docs/compatibility/8.0/android-8.0-cdd> Retrieved 12.01.2025.
- [3] Android Key Attestation Library 2024. Retrieved 04.01.2025. <https://developer.android.com/privacy-and-security/security-key-attestation>
- [4] Android security documentation - Verified Boot - Boot flow 2024. <https://source.android.com/docs/security/features/verifiedboot/boot-flow> Retrieved 10.01.2025.
- [5] AOSP - Compatibility Test Suite 2024. <https://source.android.com/docs/compatibility/cts> Retrieved 12.01.2025.
- [6] Attestation compatibility guide 2024. <https://grapheneos.org/articles/attestation-compatibility-guide> Retrieved 12.01.2025.
- [7] Attestation.app 2024. <https://attestation.app/> Retrieved 12.01.2025.
- [8] Claims about the anti-competitive nature of Play Integrity 2024. <https://web.archive.org/web/20240731163148/https://nitter.privacydev.net/GrapheneOS/status/1818415391791570995> Retrieved 12.01.2025.
- [9] Muhammad Ibrahim, Abdullah Imran, and Antonio Bianchi. 2021. SafetyNOT: on the usage of the SafetyNet attestation API in Android. In *Proceedings of the 19th Annual International Conference on Mobile Systems, Applications, and Services (Virtual Event, Wisconsin) (MobiSys '21)*. Association for Computing Machinery, New York, NY, USA, 150–162. <https://doi.org/10.1145/3458864.3466627>
- [10] Inside Safetynet 2015. Retrieved 04.01.2025. <https://koz.io/inside-safetynet/>
- [11] Magisk 2024. Retrieved 18.12.2024. <https://github.com/topjohnwu/Magisk>
- [12] Play Integrity Documentation 2024. <https://developer.android.com/google/play/integrity> Retrieved 10.1.2025.
- [13] Play Integrity Documentation - Integrity Verdicts 2024. <https://developer.android.com/google/play/integrity/verdicts#device-integrity-field> Retrieved 10.1.2025.
- [14] Universal SafetyNet Fix 2023. . <https://github.com/kdrag0n/safetynet-fix> Retrieved 18.12.2024.